Week 8 · Reports and screens

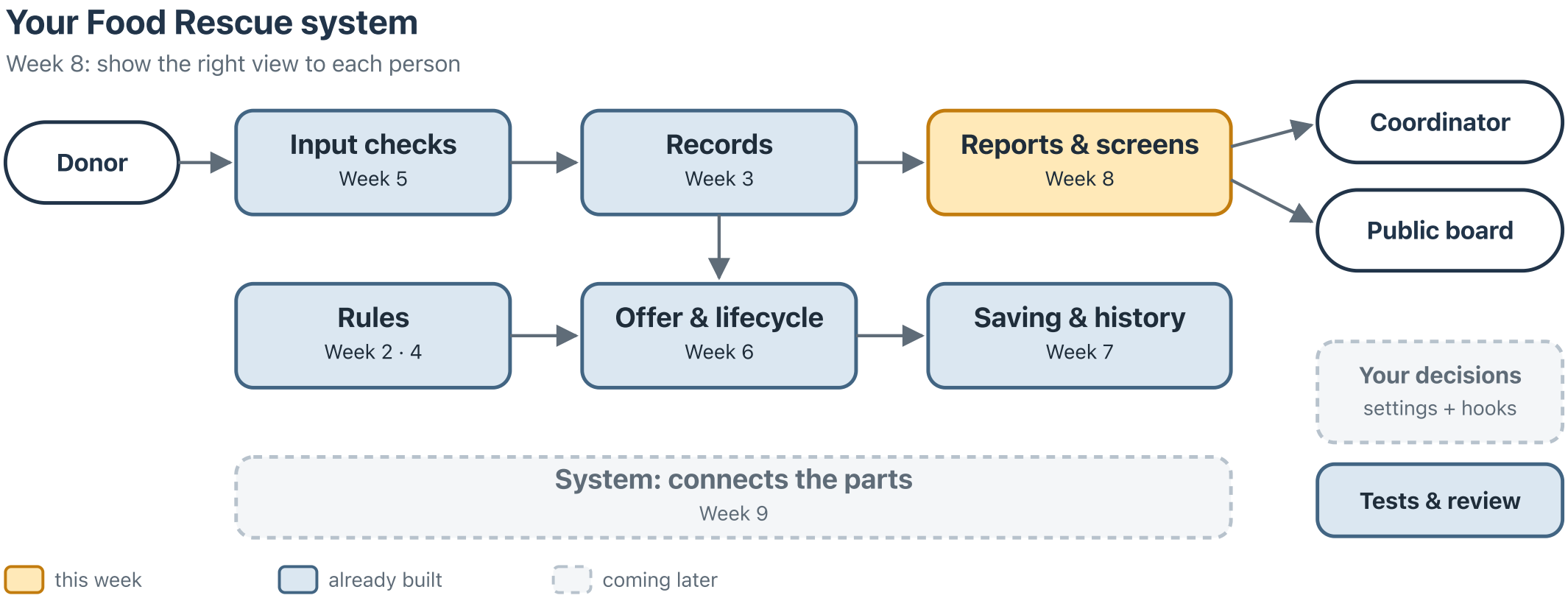
Programming for Information Systems

The right information for the right person

This week

Question	Who needs which information? How do we keep numbers honest and private data private?
Python	`sorted()` with a key function, choosing fields, lined-up f-strings, a menu function
Principle 8	Right information for the right person
You build	`public_row`, `most_urgent_first`, `handle_choice`

Your system, by Week 10



Same data, different views

View	Who reads it	Needs	Must not show
Coordinator view	the coordinator	every open offer, the donor's **contact**	nothing is hidden
Public board	anyone	what food is moving, how much	contact details

> A hospital: the doctor sees your file; the waiting-room screen shows only your ticket number.

Honest numbers

```
Portions offered:       100
Portions picked up:      48
Portions delivered:      18
```

- Three **stages** of the same food: F002's 18 portions are in all three.
- **Never add them together.** 166 counts the same portions three times.
- The unit is **portions**, not "meals".

Python foundations

Order, fields and text

`sorted()` makes a new list

```
marks = [72, 45, 90, 61]
print(sorted(marks))                # [45, 61, 72, 90]
print(sorted(marks, reverse=True))  # [90, 72, 61, 45]
print(marks)                        # [72, 45, 90, 61]  unchanged
```

- The original list does not change.
- Two views can sort the same records in different orders.

Sorting records: a named key function

```
def by_mark(student):
    return student["mark"]

for student in sorted(students, key=by_mark):
    print(student["name"], student["mark"])
```

- The key function gets **one** item and returns the value to sort by.
- `key=by_mark` **without** `( )`: you hand over the function itself.

Choosing fields: an allowed list

```
NOTICE_BOARD_FIELDS = ["name", "class"]

def notice_board_row(student):
    row = {}
    for field in NOTICE_BOARD_FIELDS:
        row[field] = student[field]
    return row
```

- Start empty, copy only what is **allowed**.
- A new private field added later stays hidden automatically.

Lining up columns

```
food = "Bread rolls"
portions = 30
print(f"{food:<12} {portions:>4} portions")
# Bread rolls    30 portions
```

- `<` left, `>` right; the number is the width.
- Text on the left, numbers on the right, so digits line up.

A menu function returns text

```
def library_answer(choice):
    if choice == "1":
        return "Open 8:00–17:00, Monday to Saturday."
    if choice == "q":
        return "Goodbye!"
    return "Unknown choice " + repr(choice) + "."
```

- **Asking** (the loop) and **deciding** (this function) are kept apart.
- A function that returns text can be tested without typing.

Food rescue: urgent food first

```
def by_hours_left(record):
    return record["hours_left"]

for record in sorted(fr.sample_records(), key=by_hours_left):
    hours = record["hours_left"]
    print(f"{record['id']:<5} {record['food']:<18} {hours:>3} hours left")
```

```
F002  Fried rice boxes      2 hours left
F004  Vegetable curry       4 hours left
```

⚖️ Good or risky? An AI's public dashboard

Version A (from an AI assistant)

```
Meals served: 148
Bananas              40  greenstall@example.org
Bread rolls          30  sunrise@example.org
Fried rice boxes     18  canteen@example.org
Vegetable curry      12  lotus@example.org
```

Version B

```
Portions delivered so far: 18
Fried rice boxes     18 portions  delivered
Vegetable curry      12 portions  approved
Bread rolls          30 portions  picked_up
```

Both run on the four sample records. Which would you put on the canteen screen?

What is wrong with Version A?

```
lines = [f"Meals served: {offered + picked_up + delivered}"]
for record in sorted(records, key=by_food_name):
    lines.append(f"{record['food']:<18} {record['portions']:>4}  {record['contact']}")
```

- Find **three** problems. For each one: which offer or person is harmed?

Principle 8 · Right information for the right person

- Each view is designed for **one person and one task**.
- **Private data stays private:** build views from an allowed list.
- **Honest numbers:** one measure, one unit; never add different measures.

> When an AI gives you a dashboard, ask: *what does this number measure, and could someone check it?*

You build: the public board and the order

```
PUBLIC_FIELDS = ["id", "food", "category", "portions", "status"]

def public_row(record):
    row = {}
    for field in PUBLIC_FIELDS:
        row[field] = record[field]
    return row

def most_urgent_first(record):
    return record["hours_left"]
```

You build: the menu's answers

```
def handle_choice(choice, system):
    if choice == "1":
        return system.report("coordinator")
    if choice == "2":
        return system.report("public")
    # "3" summary_text, "4" history_text, "q" "Goodbye!"
    return "Unknown choice " + repr(choice) + ". Please type 1–6 or q."
```

- **Return** the text; the kit's loop prints it.

Plug in

```
MY_SETTINGS = {"PUBLIC_FIELDS": PUBLIC_FIELDS, "REPORT_SORT": REPORT_SORT}
my_system = fr.System(MY_SETTINGS, hooks={"public_row": public_row})
my_system.run_day(show=False)
print(my_system.report("public"))

sorted(my_system.records(), key=most_urgent_first)
fr.run_menu(my_system, script=["1", "2", "q"], handle=handle_choice)
# fr.run_menu(my_system, handle=handle_choice)    # the real menu: remove the #
```

🧭 Your decision · who sees what, in which order?

- `PUBLIC_FIELDS`: a donor may **want** public recognition, but the **contact** must stay private.
- `REPORT_SORT`: `"urgency"`, `"portions"` or `"id"`.

Decision log	
My choice	`PUBLIC_FIELDS = …` · `REPORT_SORT = …`
Who is affected	who is protected, who sees less

This week's tasks

- ⭐ **Core:** P1–P4, Good or risky, the three functions with all ✅, the decision log.
- ⭐⭐ **Stretch:** sort by two things; a "v" menu choice for volunteers.
- No new research task: Research 6 was the last one.
- Weeks 9 and 10 focus on the **final project**.

Next week · Putting the system together

Every part you built, in **one** system, with **all** your settings. You start your final project.